

Fatima Jinnah Women University



Cloud Computing

Assignment 2

Multi Tier Web Infrastructure (Terraform + AWS)

Submitted by:

Inshal Nasir

Roll No:

38

Section :

A

Submitted to:

Sir . Waqas

1.1 Project Structure

Description:

A modular Terraform project structure was created following best practices to separate networking, security, and compute resources.

```
.:
README.md  locals.tf  modules/  screenshots/  terraform.tfvars
docs/      main.tf    outputs.tf  scripts/      variables.tf

./docs:

./modules:
networking/  security/  webserver/

./modules/networking:
main.tf  outputs.tf  variables.tf

./modules/security:
main.tf  outputs.tf  variables.tf

./modules/webserver:
main.tf  outputs.tf  variables.tf

./screenshots:
bonus/  part1/  part2/  part3/  part4/  part5/  part6/

./screenshots/bonus:

./screenshots/part1:

./screenshots/part2:

./screenshots/part3:

./screenshots/part4:

./screenshots/part5:

./screenshots/part6:

./scripts:
apache-setup.sh  nginx-setup.sh
```

- assignment_part1_project_structure.png

```
$ cat .gitignore
# Terraform files
.terraform/
terraform.tfstate
terraform.tfstate.backup
*.tfvars

# SSH Keys
*.pem
*.key

# OS files
.DS_Store
```

- assignment_part1_gitignore.png

1.2 Variable Configuration

Description:

All required variables were defined in variables.tf with descriptions and proper defaults. Sensitive values were excluded using .gitignore

```
$ cat variables.tf
variable "vpc_cidr_block" {
  description = "CIDR block for VPC"
  type       = string
}

variable "subnet_cidr_block" {
  description = "CIDR block for subnet"
  type       = string
}

variable "availability_zone" {
  description = "Availability zone"
  type       = string
}

variable "env_prefix" {
  description = "Environment prefix"
  type       = string
  default    = "prod"
}

variable "instance_type" {
  description = "EC2 instance type"
  type       = string
  default    = "t4g.micro"
}

variable "public_key" {
  description = "SSH public key path"
  type       = string
}

variable "private_key" {
  description = "SSH private key path"
  type       = string
}

variable "instance_name" {
  type       = string
  description = "Instance base name"
}

variable "instance_suffix" {
  type       = string
  description = "Unique instance suffix"
}
```

```

variable "instance_suffix" {
  type      = string
  description = "Unique instance suffix"
}

variable "subnet_id" {
  type      = string
  description = "Subnet ID"
}

variable "vpc_id" {
  type      = string
  description = "VPC ID"
}

variable "security_group_id" {
  type      = string
  description = "Security group ID"
}

variable "script_path" {
  type      = string
  description = "Path to user-data script"
}

variable "common_tags" {
  type      = map(string)
  description = "Common resource tags"
}

```

- assignment_part1_variables_tf.png

```

# VPC & Subnet
vpc_cidr_block      = "10.0.0.0/16"
subnet_cidr_block  = "10.0.10.0/24"
availability_zone   = "me-central-1"

# Environment & Instance
env_prefix          = "prod"
instance_name       = "web"
instance_suffix     = "01"
instance_type       = "t4g.micro"

# Network & Security
vpc_id              = "vpc-079a26ed9b9cb32f2" # replace with your VPC ID
subnet_id           = "subnet-03c38a2393aaf6920" # replace with your Subnet ID
security_group_id   = "sg-0cb37394bf16ab2a5" # replace with your Security Group ID

# SSH keys
public_key          = "C:/Users/Alli/.ssh/id_ed25519_new.pub"
private_key         = "C:/Users/Alli/.ssh/id_ed25519_new"

# Scripts & Tags
script_path         = "nginx-setup.sh"

common_tags = {
  Project = "MyProject"
  Owner   = "YourName"
}

```

- assignment_part1_terraform_tfvars.png

1.3 Networking Module

Description:

A reusable networking module was implemented to provision:

- VPC
- Public subnet
- Internet Gateway
- Route Table and associations

```
$ cat modules/networking/main.tf
resource "aws_vpc" "this" {
  cidr_block = var.vpc_cidr_block
  tags = {
    Name = "${var.env_prefix}-vpc"
  }
}

resource "aws_subnet" "this" {
  vpc_id            = aws_vpc.this.id
  cidr_block        = var.subnet_cidr_block
  availability_zone  = var.availability_zone
  map_public_ip_on_launch = true

  tags = {
    Name = "${var.env_prefix}-subnet"
  }
}

resource "aws_internet_gateway" "this" {
  vpc_id = aws_vpc.this.id

  tags = {
    Name = "${var.env_prefix}-igw"
  }
}

resource "aws_route_table" "this" {
  vpc_id = aws_vpc.this.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.this.id
  }

  tags = {
    Name = "${var.env_prefix}-rt"
  }
}

resource "aws_route_table_association" "this" {
  subnet_id      = aws_subnet.this.id
  route_table_id = aws_route_table.this.id
}
```

- assignment_part1_networking_module_main.png

```
$ cat modules/networking/outputs.tf
output "vpc_id" {
  value = aws_vpc.this.id
}

output "subnet_id" {
  value = aws_subnet.this.id
}

output "igw_id" {
  value = aws_internet_gateway.this.id
}

output "route_table_id" {
  value = aws_route_table.this.id
}
```

- assignment_part1_networking_module_outputs.png

1.4 Security Module

Description:

Security groups were created to restrict access:

- SSH access allowed only from user IP
- HTTP/HTTPS allowed on Nginx
- Backend servers accessible only from Nginx

```
$ cat modules/security/variables.tf
variable "vpc_id" {
  type = string
}

variable "env_prefix" {
  type = string
}

variable "my_ip" {
  type = string
}
```

- assignment_part1_security_module.png

```
ATI1@DESKTOP-KQE4P80 MINGW64 ~/Lab12_Assignment (master)
```

```
$ cat modules/security/main.tf
```

```
resource "aws_security_group" "nginx" {
  name     = "${var.env_prefix}-nginx-sg"
  vpc_id   = var.vpc_id

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = [var.my_ip]
  }

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port = 443
    to_port   = 443
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_security_group" "backend" {
  name     = "${var.env_prefix}-backend-sg"
  vpc_id   = var.vpc_id

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    security_groups = [aws_security_group.nginx.id]
  }
}
```

```
$ cat modules/security/outputs.tf

output "nginx_sg_id" {
  value = aws_security_group.nginx.id
}

output "backend_sg_id" {
  value = aws_security_group.backend.id
}
```

- assignment_part1_security_groups_console.png

1.5 Locals Configuration

Description:

Terraform locals were used to define reusable values such as tags, backend server list, and dynamic IP detection.

```
Alii@DESKTOP-KQE4P80 MINGW64 ~/Lab12_Assignment (master)
$ cat locals.tf
data "http" "my_ip" {
  url = "https://icanhazip.com"
}

locals {
  my_ip = "${chomp(data.http.my_ip.response_body)}/32"

  common_tags = {
    Environment = var.env_prefix
    Project      = "Lab12-Assignment"
    ManagedBy    = "Terraform"
  }

  backend_servers = [
    {
      name       = "web-1"
      suffix     = "1"
      script_path = "./scripts/apache-setup.sh"
    },
    {
      name       = "web-2"
      suffix     = "2"
      script_path = "./scripts/apache-setup.sh"
    },
    {
      name       = "web-3"
      suffix     = "3"
      script_path = "./scripts/apache-setup.sh"
    }
  ]
}
```

- assignment_part1_locals_tf.png

2.1 Webserver Module Design

Description:

A reusable EC2 module was created to deploy both Nginx and backend servers with configurable parameters

ATTI@DESKTOP-KQE4P80 MINGW64 ~/Lab12_Assignment (master)

\$ cat modules/webserver/variables.tf

```
variable "env_prefix" {
  type      = string
  description = "Environment prefix"
}

variable "instance_name" {
  type      = string
  description = "Instance base name"
}

variable "instance_type" {
  type      = string
  description = "EC2 instance type"
}

variable "availability_zone" {
  type      = string
  description = "Availability zone"
}

variable "vpc_id" {
  type      = string
  description = "VPC ID"
}

variable "subnet_id" {
  type      = string
  description = "Subnet ID"
}

variable "security_group_id" {
  type      = string
  description = "Security group ID"
}

variable "public_key" {
  type      = string
  description = "Public key material"
}

variable "script_path" {
  type      = string
  description = "Path to user-data script"
}
```

- assignment_part2_webserver_module_variables.png

```
$ cat main.tf
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["a12023-ami-*-x86_64"]
  }
}

resource "aws_key_pair" "this" {
  key_name   = "${var.env_prefix}-${var.instance_name}-${var.instance_suffix}"
  public_key = file(var.public_key)
}

resource "aws_instance" "this" {
  ami                  = data.aws_ami.amazon_linux.id
  instance_type        = var.instance_type
  availability_zone     = var.availability_zone
  subnet_id           = var.subnet_id
  vpc_security_group_ids = [var.security_group_id]
  key_name             = aws_key_pair.this.key_name
  associate_public_ip_address = true

  user_data = file(var.script_path)

  tags = merge(
    var.common_tags,
    {
      Name = "${var.env_prefix}-${var.instance_name}"
      Role = var.instance_name
    }
  )
}
```

- assignment_part2_webserver_module_main.png

```
$ cat outputs.tf
output "instance_id" {
  description = "EC2 instance ID"
  value       = aws_instance.this.id
}

output "public_ip" {
  description = "Public IP address"
  value       = aws_instance.this.public_ip
}

output "private_ip" {
  description = "Private IP address"
  value       = aws_instance.this.private_ip
}
```

- assignment_part2_webserver_module_outputs.png

2.2 Module Usage

Description:

The webserver module was instantiated:

- Once for Nginx
- Multiple times for backend servers using for_each

```
# Set hostname
hostnamectl set-hostname backend-server

# Create HTML page
cat > /var/www/html/index.html <<EOF
<!DOCTYPE html>
<html>
<head>
  <title>Backend Web Server</title>
  <style>
    body {
      font-family: Arial;
      background: linear-gradient(135deg, #667eea, #764ba2);
      color: white;
      padding: 40px;
    }
    .box {
      background: rgba(255,255,255,0.15);
      padding: 25px;
      border-radius: 10px;
    }
    .label {
      font-weight: bold;
      color: #ffd700;
    }
  </style>
</head>
<body>
  <div class="box">
    <h1>Backend Server - Lab 12</h1>
    <p><span class="label">Hostname:</span> $(hostname)</p>
    <p><span class="label">Instance ID:</span> $INSTANCE_ID</p>
    <p><span class="label">Private IP:</span> $PRIVATE_IP</p>
    <p><span class="label">Public IP:</span> $PUBLIC_IP</p>
    <p><span class="label">Public DNS:</span> $PUBLIC_DNS</p>
    <p><span class="label">Deployed At:</span> $(date)</p>
    <p><span class="label">Status:</span> ☒ Active</p>
  </div>
</body>
</html>
EOF

chmod 644 /var/www/html/index.html

echo "Apache backend setup completed successfully"
```

Part 3: Server Configuration Scripts

3.1 Apache Backend Server Script

Description:

A bash script was created to:

- Install Apache
- Display server metadata
- Serve a custom HTML page

```

Alli@DESKTOP-KQE4P80 MINGW64 ~/Lab12_Assignment (master)
$ cat nginx-setup.sh
#!/bin/bash
set -e

# Update system and install Nginx
yum update -y
yum install -y nginx openssl

systemctl start nginx
systemctl enable nginx

# Create SSL directories
mkdir -p /etc/ssl/private /etc/ssl/certs

# Get IMDSv2 token
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
-H "X-aws-ec2-metadata-token-ttl-seconds: 21600")

# Get public IP
PUBLIC_IP=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/public-ipv4)

# Generate self-signed certificate
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout /etc/ssl/private/selfsigned.key \
-out /etc/ssl/certs/selfsigned.crt \
-subj "/CN=$PUBLIC_IP" \
-addext "subjectAltName=IP:$PUBLIC_IP"

# Backup original config
cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.bak

# Write nginx config
cat > /etc/nginx/nginx.conf <<'EOF'
user nginx;
worker_processes auto;
error_log /var/log/nginx/error.log;
pid /run/nginx.pid;

events {
    worker_connections 1024;
}

http {
    include /etc/nginx/mime.types;
    default_type application/octet-stream;

```

- assignment_part3_apache_script.png

```

upstream backend_servers {
    server BACKEND_IP_1:80;
    server BACKEND_IP_2:80;
    server BACKEND_IP_3:80 backup;
}

server {
    listen 443 ssl;
    server_name _;

    ssl_certificate      /etc/ssl/certs/selfsigned.crt;
    ssl_certificate_key  /etc/ssl/private/selfsigned.key;

    add_header X-Frame-Options SAMEORIGIN;
    add_header X-Content-Type-Options nosniff;
    add_header X-XSS-Protection "1; mode=block";

    location / {
        proxy_pass http://backend_servers;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;

        proxy_cache mycache;
        proxy_cache_valid 200 60m;
        add_header X-Cache-Status $upstream_cache_status;
    }

    server {
        listen 80;
        server_name _;
        return 301 https://$host$request_uri;
    }
}

```

OF

```

# Create cache directory
mkdir -p /var/cache/nginx
chown -R nginx:nginx /var/cache/nginx

# Test and restart nginx
nginx -t && systemctl restart nginx

echo "Nginx setup completed successfully"

```

- assignment_part3_backend_webpage.png

4.1 – Infrastructure Deployment (5 marks)

```
$ terraform fmt
terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Finding latest version of hashicorp/http...
- Installing hashicorp/http v3.5.0...
- Installed hashicorp/http v3.5.0 (signed by HashiCorp)
- Installing hashicorp/aws v6.27.0...
- Installed hashicorp/aws v6.27.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.
```

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

- assignment_part4_terraform_init.png

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

Success! The configuration is valid.

- assignment_part4_terraform_validate.png

```
data.http.my_ip: Reading...
data.aws_ami.amazon_linux: Reading...
aws_key_pair.this: Refreshing state... [id=prod-web-01]
data.http.my_ip: Read complete after 1s [id=https://icanhazip.com]
data.aws_ami.amazon_linux: Read complete after 1s [id=ami-075212746751066bd]
aws_instance.this: Refreshing state... [id=i-0df859bb87a931c9f]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

# aws_key_pair.this must be replaced
-/+ resource "aws_key_pair" "this" {
  - arn          = "arn:aws:ec2:us-central-1:623705167686:key-pair/prod-web-01" -> (known after apply)
  - fingerprint = "dc7WNGAO1Wb0n1d1KXTFb2ThMR3ViiJDVwglckKMe" -> (known after apply)
  - id          = "prod-web-01" -> (known after apply)
  + key_name_prefix = (known after apply)
  + key_pair_id    = "key-0cf9c1fca39440d8d" -> (known after apply)
  - key_type      = "ed25519" -> (known after apply)
  + public_key    = "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAI798Gyp/ro34ZKiohF7zoidIDodMgFNvp1wmlTA6Fmd All1@DESKTOP-KQE4P80" -> "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIPGqj8006WooSgrrvX5i0KDGU8fzqd1/8HomZvg7Hc inshalinshalin@gmail.com" # forces replacement
  - tags         = {} -> null
  - tags_all     = {} -> (known after apply)
  # (2 unchanged attributes hidden)
}

Plan: 1 to add, 0 to change, 1 to destroy.
```

- assignment_part4_terraform_plan.png

```
terraform apply
data.http.my_ip: Reading...
data.http.my_ip: Read complete after 0s [id=https://icanhazip.com]
data.aws_ami.amazon_linux: Reading...
aws_key_pair.this: Refreshing state... [id=prod-web-01]
data.aws_ami.amazon_linux: Read complete after 1s [id=ami-009ce8169fc88edf5]
```

- assignment_part4_terraform_apply.png

4.2 Terraform Outputs

Description:

Terraform outputs were configured to display:

- Instance IDs
- Public and private IPs
- Backend server information

Outputs :

```
instance_id = "i-0673b0ffe1ae7ffe4"
private_ip  = "172.31.12.186"
public_ip   = "3.28.240.200"
```

- assignment_part4_terraform_output.png

4.3 AWS Console Verification

Description:

All resources were verified in the AWS Console to ensure correct deployment

The screenshot shows the AWS Management Console VPC dashboard for a VPC in the Middle East (UAE) region. The VPC ID is vpc-079a26ed9b9cb32f2 and its state is Available. The dashboard displays various configuration details in a grid:

- VPC ID:** vpc-079a26ed9b9cb32f2
- State:** Available
- Block Public Access:** Off
- DNS hostnames:** Enabled
- DNS resolution:** Enabled
- Tenancy:** default
- DHCP option set:** dopt-02dd4ed017908b331
- Main network ACL:** acl-0f1566c783d56e6fc
- Default VPC:** Yes
- IPv4 CIDR:** 172.31.0.0/16
- Main route table:** rtb-08fae696c7aea3f9d
- IPv6 CIDR:** -
- Route 53 Resolver DNS Firewall rule groups:** -
- IPv6 pool:** -
- Encryption control ID:** -
- Encryption control mode:** -
- Owner ID:** 623705167686

Below the grid, the 'CIDRs' tab is selected, showing the IPv4 CIDR 172.31.0.0/16 with a status of 'Associated'. The left sidebar shows the navigation menu with 'Your VPCs' selected. The bottom of the screen shows the Windows taskbar with various application icons and the system clock indicating 12:41 AM on 12/26/2025.

SS in Document Q

Google Gemini

Free Cloud Comp

AWS Management

VPC | me-central-1

New Tab

me-central-1.console.aws.amazon.com/vpconsole/home?region=me-central-1#SubnetDetails:subnetId=subnet-03c38a2393aaf6920

aws

Search

[Alt+S]

Middle East (UAE)

Account ID: 6237-0516-7686

inshal nasir

VPC > Subnets > subnet-03c38a2393aaf6920

VPC dashboard

AWS Global View

Filter by VPC

Virtual private cloud

Your VPCs

Subnets

Route tables

Internet gateways

Egress-only internet gateways

DHCP option sets

Elastic IPs

Managed prefix lists

Endpoints

Endpoint services

Details

Subnet ID

subnet-03c38a2393aaf6920

IPv4 CIDR

172.31.0.0/20

Availability Zone

mec1-az3 (me-central-1c)

Default subnet

Yes

IPv4 CIDR reservations

-

Resource name DNS A record

Disabled

Subnet ARN

arn:aws:ec2:me-central-1:623705167686:subnet/subnet-03c38a2393aaf6920

Available IPv4 addresses

4090

VPC

vpc-079a26ed9b9cb32f2

Auto-assign public IPv4 address

Yes

IPv6 CIDR reservations

-

Resource name DNS AAAA record

Disabled

State

Available

IPv6 CIDR

-

Route table

rtb-08fae696c7aea3f9d

Auto-assign IPv6 address

No

IPv6-only

No

DNS64

Disabled

Block Public Access

Off

IPv6 CIDR association ID

-

Network ACL

acl-0f1566c783d56e6fc

Outpost ID

-

Hostname type

IP name

Owner

623705167686

CloudShell

Feedback

Console Mobile App

© 2025, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

12:43 AM

12/26/2025

SS in Document Q

Google Gemini

Free Cloud Comp

AWS Management

VPC | me-central-1

New Tab

me-central-1.console.aws.amazon.com/vpconsole/home?region=me-central-1#InternetGateway:internetGatewayId=igw-029437efa8c7ac563

aws

Search

[Alt+S]

Middle East (UAE)

Account ID: 6237-0516-7686

inshal nasir

VPC > Internet gateways > igw-029437efa8c7ac563

VPC dashboard

AWS Global View

Filter by VPC

Virtual private cloud

Your VPCs

Subnets

Route tables

Internet gateways

Egress-only internet gateways

DHCP option sets

Elastic IPs

Managed prefix lists

Endpoints

Endpoint services

igw-029437efa8c7ac563

Details

Internet gateway ID

igw-029437efa8c7ac563

State

Attached

VPC ID

vpc-079a26ed9b9cb32f2

Owner

623705167686

Tags

Search tags

Key

Value

No tags associated with this resource

Manage tags

CloudShell

Feedback

Console Mobile App

© 2025, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

12:43 AM

12/26/2025

me-central-1.console.aws.amazon.com/vpcconsole/home?region=me-central-1#RouteTableDetails:RouteTableId=rtb-08fae696c7aea3f9d

Account ID: 6237-0516-7686 inshal nasir

VPC > Route tables > rtb-08fae696c7aea3f9d

Virtual private cloud

- Your VPCs
- Subnets
- Route tables**
- Internet gateways
- Egress-only internet gateways
- DHCP option sets
- Elastic IPs
- Managed prefix lists
- Endpoints
- Endpoint services
- NAT gateways
- Peering connections
- Route servers

Security

Details Info

Route table ID: rtb-08fae696c7aea3f9d

VPC: vpc-079a26ed9b9cb32f2

Main: Yes

Owner ID: 623705167686

Explicit subnet associations: -

Edge associations: -

Routes Subnet associations Edge associations Route propagation Tags

Routes (2) Both Edit routes

Filter routes

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	igw-029437efa8c7ac5...	Active	No	Create Route
172.31.0.0/16	local	Active	No	Create Route Table

CloudShell Feedback Console Mobile App © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

12:44 AM 12/26/2025

Task 5

```
ssh-keygen -t ed25519 -f /c/Users/Alli/.ssh/nginx_key
Generating public/private ed25519 key pair.
Enter passphrase for "/c/Users/Alli/.ssh/nginx_key" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /c/Users/Alli/.ssh/nginx_key
Your public key has been saved in /c/Users/Alli/.ssh/nginx_key.pub
The key fingerprint is:
SHA256:rdKwO4JaoVfrP/XTIV5HEdnNJov3DBGbxwRnc1jhqb4 Alli@DESKTOP-KQE4P80
The key's randomart image is:
--[ED25519 256]--+
  *ΛO|
  B=&|
  oB+|
  o.|
  . . . S . o
  . O . +... + .
  . O.. O.OO + +
  O... OO + . .
  .. .OOO . E
```

```
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '3.28.240.200' (ED25519) to the list of known hosts.
Enter passphrase for key 'C:/Users/Alli/.ssh/id_ed25519_new':
```

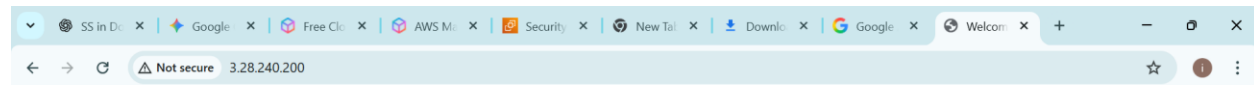


Amazon Linux 2023 (ECS Optimized)

```
For documentation, visit http://aws.amazon.com/documentation/ecs
[ec2-user@ip-172-31-12-186 ~]$ |
```

```
ec2-user@ip-172-31-12-186 ~]$ sudo systemctl status nginx
● nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)
   Active: active (running) since Fri 2025-12-26 09:44:09 UTC; 4min 22s ago
     Main PID: 1914 (nginx)
       Tasks: 3 (limit: 1001)
      Memory: 3.2M
         CPU: 49ms
    CGroup: /system.slice/nginx.service
            └─1914 "nginx: master process /usr/sbin/nginx"
              └─1915 "nginx: worker process"
                └─1916 "nginx: worker process"

Dec 26 09:44:09 ip-172-31-12-186.me-central-1.compute.internal systemd[1]: Starting nginx.service - The nginx HTTP and reverse proxy server...
Dec 26 09:44:09 ip-172-31-12-186.me-central-1.compute.internal nginx[1912]: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
Dec 26 09:44:09 ip-172-31-12-186.me-central-1.compute.internal nginx[1912]: nginx: configuration file /etc/nginx/nginx.conf test is successful
Dec 26 09:44:09 ip-172-31-12-186.me-central-1.compute.internal systemd[1]: Started nginx.service - The nginx HTTP and reverse proxy server.
```



Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.



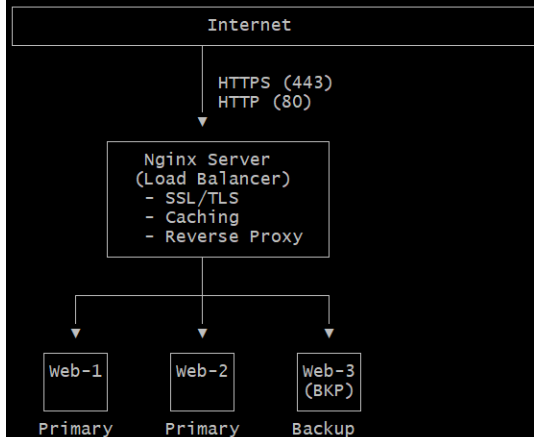
6.1 README Documentation

Description:

A comprehensive README was created including architecture, deployment steps, and troubleshooting.

```
$ cat README.md
# Assignment 2 - Multi-Tier Web Infrastructure

## Project Overview
This project deploys a multi-tier web infrastructure on AWS using Terraform.
- 1 Nginx server as a load balancer
- 3 backend servers (Web servers)
- Automatic IP and security configuration
## Architecture Diagram
```



```
## Components Description

- **Nginx Server**: Handles HTTP/HTTPS requests, load balancing, and reverse proxy to backend servers.
- **Web Servers**: Serve application content. Two primary and one backup.
- **Security Groups**: Restrict access to SSH and HTTP/HTTPS.
- **Terraform**: Manages infrastructure provisioning and teardown.
```

Prerequisites

```
## Prerequisites

Before deploying this infrastructure, make sure you have the following set up:

### Required Tools
- **Terraform**: Installed on your local machine. [Install Terraform](https://developer.hashicorp.com/terraform/downloads)
- **AWS CLI**: Installed and configured for your AWS account. [Install AWS CLI](https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html)
- **SSH Client**: To connect to EC2 instances (e.g., OpenSSH or PuTTY for Windows).

### AWS Credentials Setup
1. Create an IAM user with **AdministratorAccess** or the required permissions for EC2, VPC, and Security Groups.
2. Configure your AWS credentials locally:
    `` bash
    aws configure
    ## Deployment Instructions

Follow the steps below to deploy the infrastructure using Terraform:

1. Navigate to the project directory:
    `` bash
    cd Lab12_Assignment
    ---

## **Configuration Guide**

`` markdown
## Configuration Guide

### Backend Server Configuration
- Backend server private IPs are automatically created by Terraform.
- These IPs are referenced in the Nginx configuration for load balancing.

### Nginx Configuration
- Configuration file location:
    `` /etc/nginx/nginx.conf
- Nginx acts as a reverse proxy and forwards traffic to backend servers.

### Testing Procedures
- Open a browser and visit:
```

```

### Nginx Configuration
- Configuration file location:
  `/etc/nginx/nginx.conf`
- Nginx acts as a reverse proxy and forwards traffic to backend servers.

### Testing Procedures
- Open a browser and visit:

---

## **Architecture Details**

```markdown
Architecture Details

Network Topology
- One public EC2 instance running Nginx
- Multiple backend EC2 instances
- All instances deployed within the same VPC

Security Groups
- Nginx Security Group:
 - Allows HTTP (80)
 - Allows HTTPS (443)
 - Allows SSH (22) from user IP only
- Backend Security Group:
 - Allows traffic only from the Nginx server

Load Balancing Strategy
- Nginx uses a round-robin strategy to distribute traffic
- One backend server is configured as a backup

Troubleshooting

Common Issues
- SSH timeout: Check security group rules and your public IP
- Nginx not loading: Ensure the service is running

Useful Commands
Check Nginx status:
```bash
sudo systemctl status nginx

```

- assignment_part6_readme.png

6.2 Infrastructure Cleanup

Description:

All AWS resources were destroyed using Terraform and verified in AWS Console.

```

Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

```

- assignment_part6_terraform_destroy_prompt.png

```

aws_instance.this: Destroying... [id=i-0673b0ffe1ae7ffe4]
aws_instance.this: Still destroying... [id=i-0673b0ffe1ae7ffe4, 00m10s elapsed]
aws_instance.this: Still destroying... [id=i-0673b0ffe1ae7ffe4, 00m20s elapsed]
aws_instance.this: Still destroying... [id=i-0673b0ffe1ae7ffe4, 00m30s elapsed]
aws_instance.this: Still destroying... [id=i-0673b0ffe1ae7ffe4, 00m40s elapsed]
aws_instance.this: Still destroying... [id=i-0673b0ffe1ae7ffe4, 00m50s elapsed]
aws_instance.this: Destruction complete after 50s
aws_key_pair.this: Destroying... [id=prod-web-01]
aws_key_pair.this: Destruction complete after 1s

Destroy complete! Resources: 2 destroyed.

```

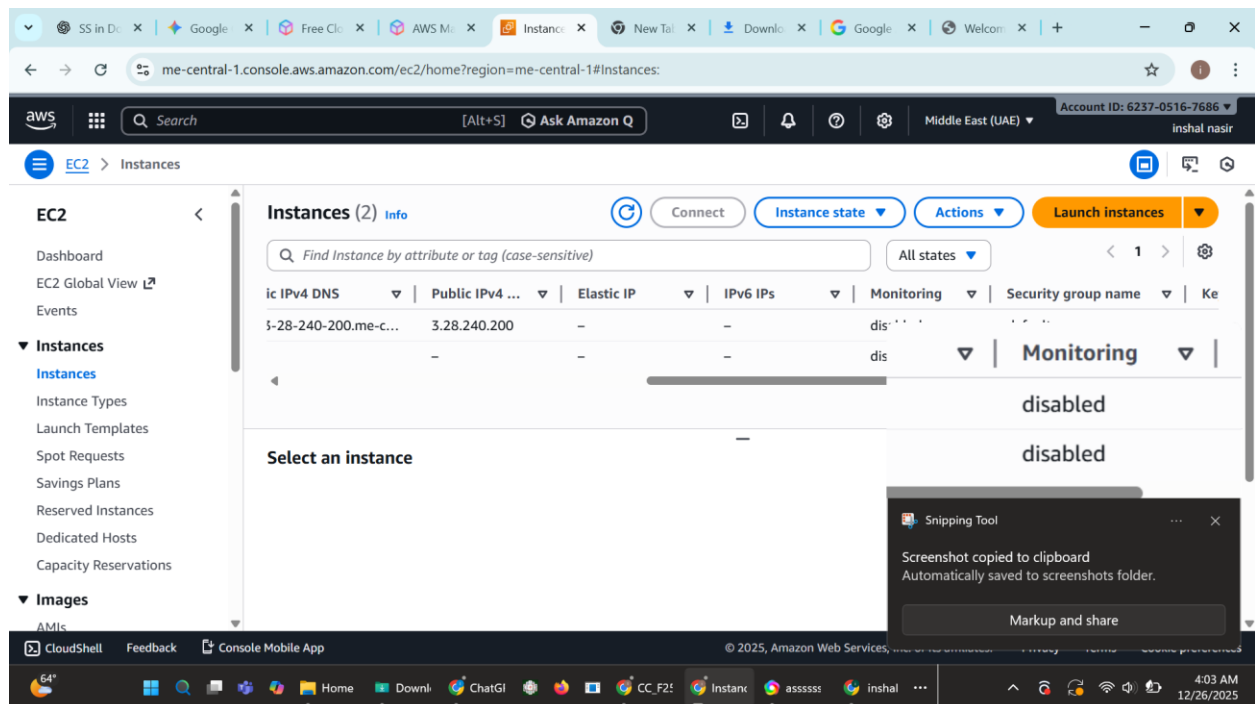
- assignment_part6_terraform_destroy_complete.png

```

$ cat terraform.tfstate
{
  "version": 4,
  "terraform_version": "1.14.3",
  "serial": 54,
  "lineage": "47443f32-2463-0c0d-5cc0-5eabae487e03",
  "outputs": {},
  "resources": [],
  "check_results": null
}

```

- assignment_part6_empty_state.png



- assignment_part6_aws_instances_destroyed.png

Conclusion

This assignment successfully demonstrated the use of Terraform to deploy a secure, scalable, and modular multi-tier web architecture on AWS, including automation, load balancing, and cleanup.
